The Open Source Audit | VITyarthi

VITyarthi | Open-Source Software

The Open-Source Audit

A Capstone Project for the OSS NGMC Course

Course: Open-Source Software |

Student Name	Registration Number
N Raja Ravi Varma	24MIM10027
Slot	Date of Submission
A24	25 March 2026

1. Introduction

A significant majority of the software powering the contemporary world—including web servers, essential developer tools, and operating systems utilized in supercomputers and mobile devices—was not developed by a single proprietary entity for commercial gain. Rather, this technology was constructed openly, disseminated without restriction, and enhanced through collective contribution, which defines the core essence of open source.

This capstone project requires a critical examination of this foundational principle.

The core requirement is to select a legitimate open-source software project—one that can be installed, researched, and utilized within a Linux environment—and execute a comprehensive, structured audit. This audit must transcend a mere functional description, focusing instead on its historical context, authorship, licensing provisions (specifically regarding user freedoms), underlying philosophical tenets, and its subsequent influence on the global technological landscape.

In addition to the formal written analysis, five distinct shell scripts must be developed to demonstrate proficiency in practical Linux administration skills. The objective of these scripts is not grand programmatic complexity, but rather to substantiate a thorough understanding of command-line open-source utilities and to illustrate the relationship between automation, sharing, and transparency.

The Relevance of this Study

Professional practice dictates that all future code development will be built upon open-source foundations. Consequently, a deep comprehension of the governing philosophy is not discretionary; it is integral to professional competence.

2. Choosing Your Software

Students are required to select one open-source project from the provided list, or alternatively, propose a different project (contingent upon instructor approval). The selected software will serve as the exclusive subject matter for the entirety of the report and all five mandatory shell scripts.

Approved software options

Software	Category	License	Why it's interesting
Linux Kernel	Operating System	GPL v2	The foundation everything else runs on
Apache HTTP Server	Web Server	Apache 2.0	Powers ~30% of all websites globally
MySQL	Database	GPL v2 / Com…	A license with a dual-model story to tell
Firefox	Web Browser	MPL 2.0	A nonprofit fighting for an open web
VLC Media Player	Multimedia	LGPL / GPL	Plays anything — built by students in Paris
LibreOffice	Office Suite	MPL 2.0	Born from a community fork — a real lesson

Python	Programming Language	PSF License	A language shaped entirely by community
Git	Version Control	GPL v2	The tool Linus built when proprietary failed him

3. Project Report Structure

The report must adhere to a length of 12 to 16 pages. Each subsequent section corresponds directly to a covered unit of the course. Content must not be artificially extended with superfluous graphical elements; instead, every page should be dedicated to original analysis and critical thought articulated in the student's own voice.

Part A — Origin and Philosophy (Units 1 & 2)

This section constitutes the most critical component of the report. A minimum of four pages must be allocated to its thorough discussion.

A1. The problem this software was created to solve

Students are strictly advised against merely duplicating the official description. Extensive research into the initial context is mandated. The analysis should delineate the specific challenges, deficiencies, or inaccessibility issues that preceded the software's creation. Furthermore, a justification must be provided for the creator's decision to pursue open dissemination rather than a proprietary model.

A2. The license — what it actually says

Obtain or locate the actual license text pertinent to your chosen software and analyze its provisions. Address the following questions within the report:

- What are the four freedoms of free software? Does your software's license grant all four?
- Can a commercial entity appropriate, modify, and sell this software without disclosing their resulting changes? Provide a rationale for your answer.
- What constitutes the differentiation between GPL and MIT licensing? Which would be the preferred choice for a personally developed project, and why?
- Is there a substantive difference between 'free as in free beer' and 'free as in freedom'? Offer a relevant illustration from your selected software.

A3. The ethics of open source

Open source transcends a simple licensing model; it is fundamentally a declaration regarding the appropriate method for sharing knowledge and instrumental tools. Provide critical reflection on the following topics:

- Should all software adhere to an open-source model? Construct compelling arguments both in support and in opposition.
- Is the creation of a profitable product entirely founded upon unpaid community labor ethically justifiable? (Consider the practices of corporations such as Red Hat, Google, and Meta in their utilization of Linux.)
- What is the meaning of the phrase 'standing on the shoulders of giants' within the context of software development? Does the open-source methodology foster or impede genuine innovation?

No predetermined correct conclusions exist for this section. Evaluation will be based solely on the intellectual rigor and depth of the presented analysis, not the resultant conclusion.

Part B — Linux Footprint (Unit 2)

Thoroughly research and document the software's residence and operation within a Linux environment. This includes its installation location, the user context under which it executes, and its management protocols. This section necessitates the actual installation of the software on a Linux system—be it a personal machine, a virtualized environment, or a designated lab system.

- Installation method — is it accessible via RPM, apt, or does it require compilation from source code?
- Key directories — where are the executable binaries, configuration files, and log files persistently stored? (e.g., `/usr/bin`, `/etc`, `/var/log`)
- User and group — what specific Linux user and group are utilized for its execution? Explain the security implications of this configuration.
- Service management — can the software be initiated, halted, or reloaded as a system service? Illustrate the relevant commands.
- Update model — how does the open-source community facilitate the deployment of security patches to the end-user machine?

Part C — The FOSS Ecosystem (Units 3 & 4)

No open-source project operates in complete isolation. Construct a map detailing the ecosystem surrounding the chosen software.

- What other open-source dependencies does it rely upon? (Libraries, compilers, runtimes)

- What subsequent developments has it enabled or inspired? (Forks, spin-offs, derived tools)
- Where does it fit within the common LAMP stack architecture, or how does it integrate with web technologies?
- Is there an established community—encompassing mailing lists, forums, conferences, or IRC channels? Identify the governing entity for this project.

Part D — Open Source vs Proprietary (Critical Analysis)

Execute a comprehensive comparison between the chosen open-source software and its nearest proprietary counterpart. The analysis must be precise and objective; avoid the assumption that open source is inherently superior.

Dimension	Open Source	Proprietary Alternative
Cost		
Security (who can audit the code?)		
Support and reliability		
Freedom to modify		
Community vs corporate control		

Your overall verdict

Complete the comparative table as an integral part of the report. Following this, construct a two-paragraph concluding statement: based on the knowledge acquired, would you formally endorse your chosen software for a practical, real-world deployment, and would you personally commit to contributing to its development?

4. Shell Script Tasks

Five shell scripts are required. Each script must be submitted as a `.sh` text file and thoroughly documented within the report, including: the script code itself, a corresponding screenshot of its execution, and a dedicated paragraph explaining its function and the specific shell scripting constructs it employs.

Important: Scripts must be demonstrably functional on a genuine Linux operating system. They must incorporate explanatory comments for each major section. Scripts lacking comments will incur a reduction in marks.

- Script 1 — System Identity Report (Unit 1 & 2)

- Script 2 — FOSS Package Inspector (Unit 2)
- Script 3 — Disk and Permission Auditor (Unit 2)
- Script 4 — Log File Analyzer (Units 2 & 5)
- Script 5 — The Open Source Manifesto Generator (Unit 5)

5. Project Timeline

Day	Focus	Tasks	Deliverable
1	Setup	Choose software. Research its origin story. Read the license text. Set up your Linux environment (VM or lab system). Install your chosen software.	Choice confirmed
2	Philosophy	Write Part A1 — the origin story. Take notes on what problem the software solved and for whom.	Draft A1
3	License & Ethics	Write Part A2 — license analysis. Write Part A3 — ethical reflection. These sections need your own thinking, not summaries.	Draft A2 + A3
4	Linux Footprint	Install software on Linux. Document directories, user, permissions. Write Part B.	Draft Part B + screenshots

5	Scripts 1 & 2	Write and test Script 1 (System Identity) and Script 2 (Package Inspector). Add them to your report with explanations.	Scripts 1–2 working
6	Scripts 3 & 4	Write and test Script 3 (Disk Auditor) and Script 4 (Log Analyzer). Test with a real log file.	Scripts 3–4 working
7	Ecosystem	Write Part C — FOSS ecosystem map. Research dependencies and community.	Draft Part C
8	Comparison & Script 5	Write Part D — comparison table and verdict. Write and test Script 5 (Manifesto Generator).	Draft Part D + Script 5
9	Report polish	Compile full report. Write introduction and conclusion. Add screenshots. Check page count (12–16 pages).	Complete draft
10	Submission	Final review. Submit report + 5 .sh files. Be ready to explain any script in a short viva if asked.	Final submission

6. Evaluation Rubric

Total marks: 100. The written report accounts for 60 marks, and the shell scripts account for 40 marks.

Report (60 marks)

Component	Full marks	Passing (50%)
1 Origin story — depth, original research, storytelling	12	6
2 License analysis — accuracy and understanding of freedoms	12	6
3 Ethical reflection — quality of argument and original thinking	10	5
4 Linux footprint — technical accuracy and screenshots	10	5
5 Ecosystem and LAMP connection	8	4
6 Open source vs proprietary comparison	8	4

Shell Scripts (40 marks — 8 marks each)

Criterion	Marks	What we look for
Correctness — does it run without errors?	4	Script executes on Linux and produces expected output
Concepts — does it use the right shell constructs?	2	Loops, if-then, case, variables used appropriately

Comments and documentation	2	Every non-obvious line has a comment; script purpose is clear

Academic integrity: All written sections must be the student's original composition.

7. Reference Resources

On open-source philosophy

- GNU Project — 'The Free Software Definition': gnu.org/philosophy/free-sw.html ● Open-Source Initiative — 'The Open Source Definition': opensource.org/osd ● Eric S. Raymond — 'The Cathedral and the Bazaar': catb.org/~esr/writings/cathedral-bazaar
- Linus Torvalds — 'Just for Fun' (autobiography, available at most libraries)

On Linux and shell scripting

- The Linux Command Line by William Shotts — freely available at linuxcommand.org
- GNU Bash Manual: gnu.org/software/bash/manual
- Linux man pages: man.cx (online) or type 'man <command>' in your terminal

For finding license texts

- SPDX License List: spdx.org/licenses
- Choose a License: choosealicense.com
- Your software's official repository on GitHub or GitLab — the LICENSE file is always there

Essential Reading on Open Source Philosophy, Linux, and Licensing

This document provides a significantly expanded list of resources for those interested in the foundational concepts of open source and free software, practical knowledge of Linux and shell scripting, and the critical topic of software licensing.-----On Open Source and Free Software Philosophy

These resources define the core tenets and historical context of the open source and free software movements, which are essential for understanding modern software development.

Resource	Description and Context	Link
The Free Software Definition	Authored by the **GNU Project** and Richard Stallman, this is the definitive statement on **Free Software** (referring to "freedom," not price). It outlines the four essential freedoms for users: the freedom to run the program as you wish, for any purpose; the freedom to study how the program works and change it; the freedom to redistribute copies; and the freedom to distribute copies of your modified versions. This definition is the philosophical bedrock for the GNU operating system and the Free Software Foundation (FSF).	gnu.org/philosophy/free-sw.html
The Open Source Definition (OSD)	Maintained by the **Open Source Initiative (OSI)**, this definition focuses on the practical criteria that must be met for a software license to be considered "open source." While sharing many principles with the FSF, the OSD's emphasis is often more pragmatic and business-focused, outlining ten key criteria including free redistribution, inclusion of source code, derived works, and non-discrimination against persons, groups, or	opensource.org/osd

	fields of endeavor.	
The Cathedral and the Bazaar	An influential essay by **Eric S. Raymond (ESR)** that contrasts two development models: the **"Cathedral"** (closed, top-down development, typical of proprietary software) and the **"Bazaar"** (open, collaborative, peer-to-peer development, characteristic of the Linux kernel). This essay coined "Linus's Law" ("Given enough eyeballs, all bugs are shallow") and helped popularize the open source model to the commercial world.	catb.org/~esr/writings/cathedral-bazaar
Just for Fun	An insightful **autobiography** by **Linus Torvalds**, the creator of the Linux kernel and Git. It offers a personal look into the mind of a pivotal figure in the open source movement, detailing the origins of Linux, the development culture it spawned, and Torvalds's pragmatic approach to software development and community management.	(Available at most libraries and bookstores)

The Myth of Innovation	**Further Reading:** A concept often discussed in open source circles, arguing that true innovation stems from sharing and collaborative building upon existing work, contrasting with the proprietary model's focus on secrecy and patent-based protection.	(Search FSF or OSI archives for related articles)

-----On Linux and Shell Scripting

These resources are vital for anyone seeking to master the command line interface, the most powerful and flexible environment for interacting with Unix-like operating systems.

Resource	Description and Context	Link
The Linux Command Line (TLCL)	By **William Shotts**, this is a comprehensive, hands-on guide that introduces beginners to the command line, starting with basic navigation and progressing to powerful concepts like stream redirection, pipelines, regular expressions, and the fundamentals of shell scripting. It is highly regarded for its clear, conversational style and is **freely available** under a Creative Commons license.	linuxcommand.org

GNU Bash Manual	The authoritative and exhaustive reference for the **GNU Bourne-Again Shell (Bash)**, the default shell on most Linux distributions and macOS. This manual details every feature, syntax rule, built-in command, and programming construct available in Bash, making it indispensable for advanced scripting and understanding shell behavior.	gnu.org/software/bash/manual
Linux Man Pages (Manual Pages)	The primary documentation system for Unix-like operating systems. Every command, system call, library function, and configuration file has an associated *man* page. Access them directly in your terminal (`man <command>`)	man.cx (online) or type `man <command>` in your terminal
	for the most accurate, system-specific documentation. The online version offers convenience.	
Advanced Bash-Scripting Guide (ABS Guide)	**Supplementary Resource:** A comprehensive guide that delves into intermediate and advanced aspects of shell scripting, covering topics such as arrays, loops, functions, environment variables, and handling signals. Essential for writing robust and complex scripts.	(Search for 'Advanced Bash-Scripting Guide')

Explain Shell	**Utility Tool:** A fantastic online resource that breaks down complex shell commands and scripts into understandable components, explaining what each argument, flag, and pipe operator does.	(Search for 'Explain Shell')

-----For Finding and Understanding License Texts

Resource	Description and Context	Link
SPDX License List	The **Software Package Data Exchange (SPDX)** is an open standard for communicating software bill of materials (SBOM) information, including licensing. The SPDX License List is an effort to standardize the naming and identification of frequently used open source and free software licenses. It is the gold standard for unambiguous license identification in automated tooling.	spdx.org/licenses

Choose a License	A highly practical, straightforward website designed to help developers quickly select a suitable open source license based on their needs, such as whether they require code contributors to share their changes (copyleft) or prefer minimal restrictions (permissive). It provides simple summaries of popular licenses like MIT, Apache 2.0, and GPL 3.0.	choosealicense.com
Official Repository License File	For any project on platforms like **GitHub** or **GitLab**, the **LICENSE** file (often a single file named `LICENSE`, `LICENSE.txt`, or similar, located in the root directory) contains the definitive, human-readable legal text governing the use of the source code within that repository. It is the authoritative source for a project's license.	Your software's official repository on GitHub or GitLab — the `LICENSE` file is always there
TLDR Legal	**Supplementary Resource:** A community-maintained site that provides concise, easy-to-understand summaries of various software licenses, translating the complex legal jargon into bullet points covering what you **can do**, what you **cannot do**, and what you **must do** under the license.	(Search for 'TLDR Legal')

GPL FAQ	**Essential Reading:** The Free Software Foundation's comprehensive Frequently Asked Questions about the **GNU General Public License (GPL)**, the most widely used copyleft license. Essential for understanding concepts like linking, aggregation, and the viral nature of copyleft.	(Search for 'GPL FAQ')